

17) Bandwidth = $\frac{\text{Highest frequency} - \text{Lowest frequency}}$

Nyquist theorem - $2 \times \text{Highest frequency}$

OS

* Variable partition:-

- In the variable partition, the allocation of the block size is not declared initially. It is declared at the time of process executing or loading.
- The first partition is reserved for OS and remaining space is divided into parts.
- The size of each partition will be equal to the size of process.
- The partition size varies according to the need of the process, so that internal fragmentation can be avoided.

* External fragmentation:

- External fragmentation means memory becomes more and more fragmented and memory utilisation declines.
- It resolve using compaction.

• Compaction:

⇒ Compaction is an OS memory management technique that rearranges fragmented, non-fragmented, non-contiguous free memory into one large-block by shifting the running processes.

⇒ It is time-consuming and wastes CPU time.

⇒ Basically, it is undesirable but still we can use that.

* Page Replacement Algorithm:-

- FIFO

- Optimal

- LRU

- MRU

* Disk Scheduling Algorithm:-

- FIFO

- SSTF

- SCAN

- C-SCAN

Paging :-

The memory management is partitioned into equal fixed size chunks and that fixed process is divided into each fixed size chunk of same size.

The chunks of the process is called pages.

The chunks or block of memory are called frames.

The operating system maintain a page table for each processes.

Process	Page	Page No	Frame No	frame
P ₀				0 page 3
P ₁		0	2	1 Page 5
P ₂		1	5	2 Page 0
P ₃		2	3	3 Page 2
P ₄		3	0	4 Page 4
P ₅		4	4	5 Page 1
P ₆		5	1	6 Page 6
		6	6	

M.M
(Main Memory)

Page Table

→ It contains the frame location for each page in process.

→ Paging is used to break the program into smaller fixed-size blocks.

* Fragmentation :-

→ Fragmentation is a memory management problem in which free memory is divided into small non-contiguous parts, resulting in inefficient utilization of memory.

Types of Fragmentation :-

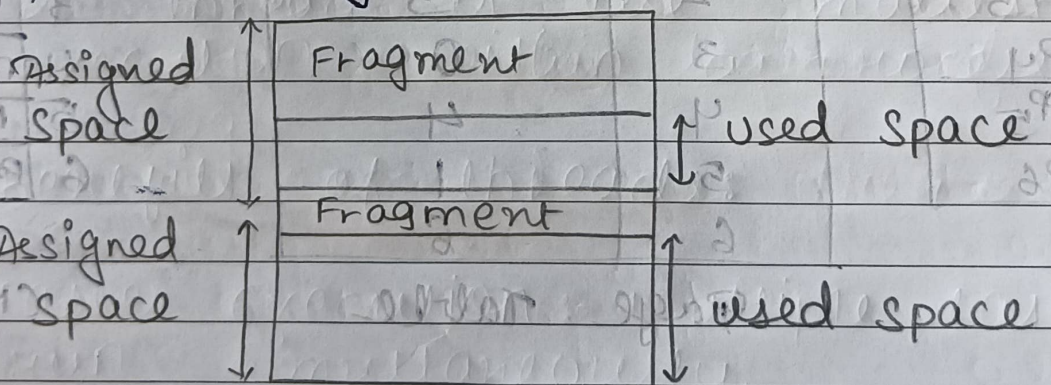
1) Internal Fragmentation

⇒ Occurs when memory is allocated in fixed-size blocks.

⇒ The allocated memory may be larger than the required memory.

⇒ The unused space within the allocated block is wasted.

⇒ Common in paging or fixed partitioning systems.

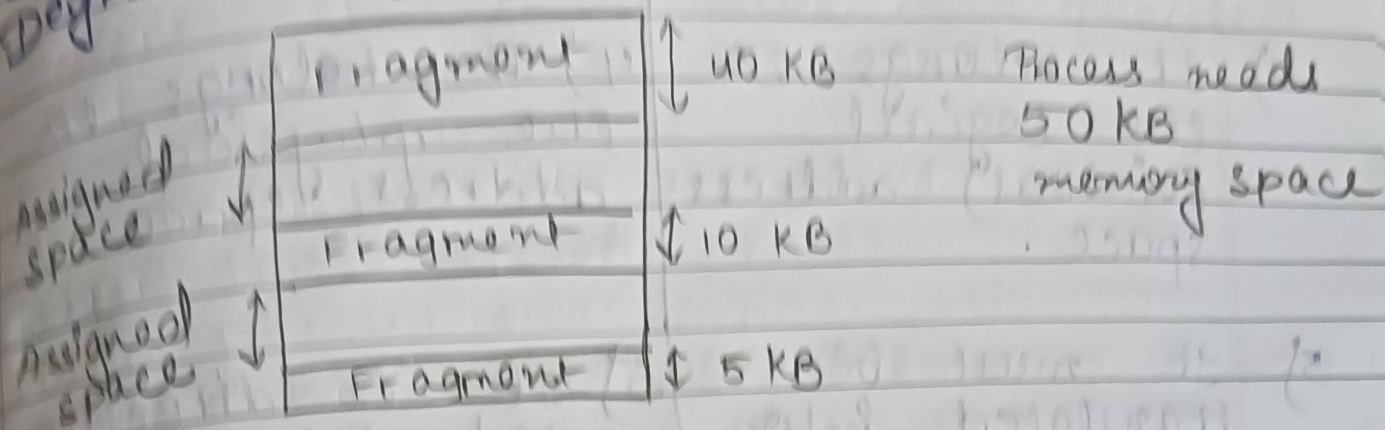


2) External Fragmentation

⇒ Occurs when free memory is divided into small, non-contiguous blocks.

⇒ Caused by frequent allocation and deallocation of processes.

total free memory may be sufficient,
but no single large block is available.
leads to inefficient memory
utilization.
Degrades system performance.



Disadvantages of Paging :-

- May cause internal fragmentation
- Requires extra memory for page tables.
- Address translation adds overhead.
- Increased complexity in memory management.

Logical Address v/s Physical Address

Logical Address	Physical Address
1) A logical address is the address generated by the CPU during program execution.	1) A physical address is the actual address in main memory (RAM).
2) It is generated by the CPU.	2) It is obtained after address translation by MMU.

3) It is visible to the user and program.

3) It is not visible to the user and is used by the memory unit.

4) It belongs to the logical (virtual) address space.

4) It belongs to the physical address space.

5) It must be translated into a physical address before accessing memory.

5) It is directly used by the memory hardware to access data.

* Threads :-

→ A thread is the smallest unit of execution within a process.

• Benefits of threads:

- Faster execution:- Multiple tasks can run simultaneously.
- Better CPU utilization:- CPU remains busy by executing threads efficiently.
- Resource sharing:- Threads share memory and resources of the same process.

Improved Responsiveness :- Applications respond quickly to users.

easy communication :- communication between threads is faster than processes.

Types of threads :-

User-Level Threads (ULT)

managed by user-level libraries instead of the operating system.
Faster to create and switch between threads.

If one thread blocks, the entire process may block.

Kernel-Level Threads (KLT)

Managed directly by the operating system kernel.
Each thread is scheduled independently by the OS.
Blocking of one thread does not affect other threads.

Multithreading :-

Multithreading is the ability of a process to execute multiple threads simultaneously.

Multiple threads work concurrently within the same process while

→ sharing memory and resources.
It improves performance, responsiveness, and CPU utilization.

* Schedulers:-

→ A scheduler is a component of the operating system that selects processes from the ready queue and allocates CPU to them for execution.

• Types of Schedulers:

1. Long-Term Scheduler

⇒ It selects processes from the job pool and loads them into main memory.

⇒ It controls the degree of multiprogramming.

⇒ It is executed less frequently.

2. Short-Term Scheduler

⇒ It selects one process from the ready queue and allocates the CPU to it.

⇒ It is executed very frequently.

⇒ It must be faster for better system performance.

Medium-Term Scheduler

- It temporarily removes processes from memory and stores them on disk (swapping).
- It helps in reducing memory load.
- It improves system performance by managing process execution.

ch-3. Inter-Process Communication

Race-condition:

- A race condition occurs when two or more processes access and modify shared data simultaneously, resulting in unpredictable and incorrect outcomes.
- It happens in systems where multiple processes execute concurrently.
- The final result depends on the order of execution of processes.

Cause of Race condition:-

- Lack of proper synchronization mechanisms.
- Simultaneous access to shared resources.
- Improper handling of critical section.

Prevention:-

Conclusion

- use semaphores or mutex locks
- allow only one process in critical section at a time.

* Mutual exclusion:-

- Mutual exclusion is a synchronization technique that ensures that only one process can access the critical section at a time.
- It prevents multiple processes from accessing shared resources simultaneously.
- It avoids data inconsistency and race conditions.
- It ensures that critical section execution is performed by only one process at a time.

• Eg:

- Two processes, P_1 and P_2 , try to access the same bank account simultaneously.
- Process P_1 starts updating the account balance.
- Using mutual exclusion, process P_2 is not allowed to access the account until P_1 completes its operation.
- After P_1 finishes, P_2 is allowed to enter the critical section and update the balance.
- This ensures correct and consistent account balance without conflicts.